جامعة مصر للمعلوماتية
EGYPT UNIVERSITY OF INFORMATICS

Digital Logic Design

Group project report

Date: May 13, 2026

Project title:

Project 3 (2-bit binary comparator)

Group members:

Habiba Mohamed 25-103411

Kenzy Ahmed 25-104205

Caren George 25-100970

Aisel Khaled 25-100880

1. Objective:

Our objective in this project is to design and implement a combinational logic circuit capable of comparing two 2-bit binary numbers (A and B).
The aim was to classify their relationship into three distinct cases: full equality (E) where inputs (A_0, A_1) are equal to inputs (B_0, B_1), a single-bit discrepancy (D) where only one bit is different in the four inputs. A total discrepancy (F) where each inputs (A_0, A_1) are completely different than inputs (B_0, B_1).

2. Truth table and Boolean expressions:

Truth table

A_0	A_1	B_0	B_1	E	D	F
0	0	0	0	1	0	0
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	0	1
0	1	0	0	0	1	0
0	1	0	1	1	0	0
0	1	1	0	0	0	1
0	1	1	1	0	1	0
1	0	0	0	0	1	0
1	0	0	1	0	0	1
1	0	1	0	1	0	0
1	0	1	1	0	1	0
1	1	0	0	0	0	1
1	1	0	1	0	1	0
1	1	1	0	0	1	0
1	1	1	1	1	0	0

K-map and Boolean expressions

K-Map for output E

A_1A_0\ B_1 B_0	00	01	11	10
00	1	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	1

Function E = $(A_1'A_0' .B_1' B_0') + (A_1'A_0.B_1' B_0) + (A_1A_0'. B_1 B_0') + (A_1A_0.B_1 B_0)$

$= (A_1 \text{ XOR } B_1)'. (A_0 \text{ XOR } B_0)' = (A_1 \text{ XNOR } B_1). (A_0 \text{ XNOR } B_0)$

K-Map for output D

A_1A_0\ B_1 B_0	00	01	11	10
00	0	1	0	1
01	1	0	1	1
11	0	1	0	1
10	1	1	1	0

Function D = $(A_1'B_1. A_0'B_0') + (A_1B_1'.A_0'B_0') + (A_1' B_1'. A_0'B_0) + (A_1'B_1.'A_0B_0')$ $+ (A_1B_1.A_0B_0) + (A_1 B_1.A_0B_0') + (A_1'B_1.A_0 B_0) + (A_1B_1'.A_0B_0) = (A_1 \text{ XOR } B_1)$ XOR $(A_0 \text{ XOR } B_0)$

K-Map for output F

A_1A_0\ B_1 B_0	00	01	11	10
00	0	0	1	0
01	0	0	0	1
11	1	0	0	0
10	0	1	0	0

Function F= $(A_1'A_0'. B_1 B_0) + (A_1'A_0. B_1 B_0') + (A_1A_0'. B_1' B_0) + (A_1A_0. B_1' B_0')$

$= (A_1 \text{ XOR } B_1). (A_0 \text{ XOR } B_0)$

3. The Logic design:

Logic diagrams

We designed the logic diagram on the software logic.ly based on the truth table and k-maps. Toggle switches were used for the inputs, XOR, AND XNOR gates and LED bulbs used for the output.

- The (F) output → $(A_1 \text{ XOR } B_1) \text{ AND } (A_0 \text{ XOR } B_0)$
- The (D) output → $(A_1 \text{ XOR } B_1) \text{ XOR } (A_0 \text{ XOR } B_0)$
- The (E) output → $(A_1 \text{ XNOR } B_1) \text{ AND } (A_0 \text{ XNOR } B_0)$

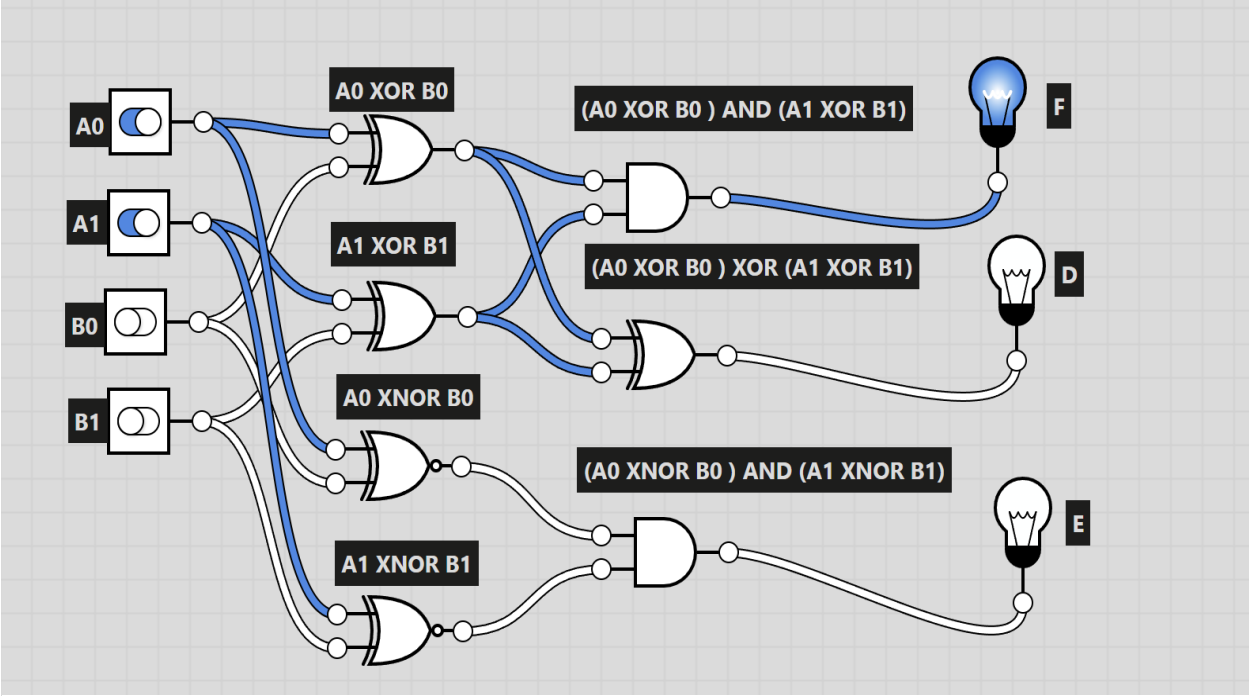


Figure 1: Logic Gate design made on Logically.

4. Implementation of the circuit:

Components used in the circuit

- Breadboard
- 6 input dip switches
- 9-volt battery
- YWRobot MB102 5-volt power supply module connected to the battery to regulate the voltage
- 4 150-ohm resistors for the dip switch and 3 330-ohm resistors for the LED bulbs
- 3 LED bulbs
- 74LS86 (XOR) gate, 74LS08(AND) gate ,74LS04(hex Inverter/NOT gate)
- Jumper wires

We first implemented the circuit with a simulator on a software called Tinkercad before trying it on a bread board directly.

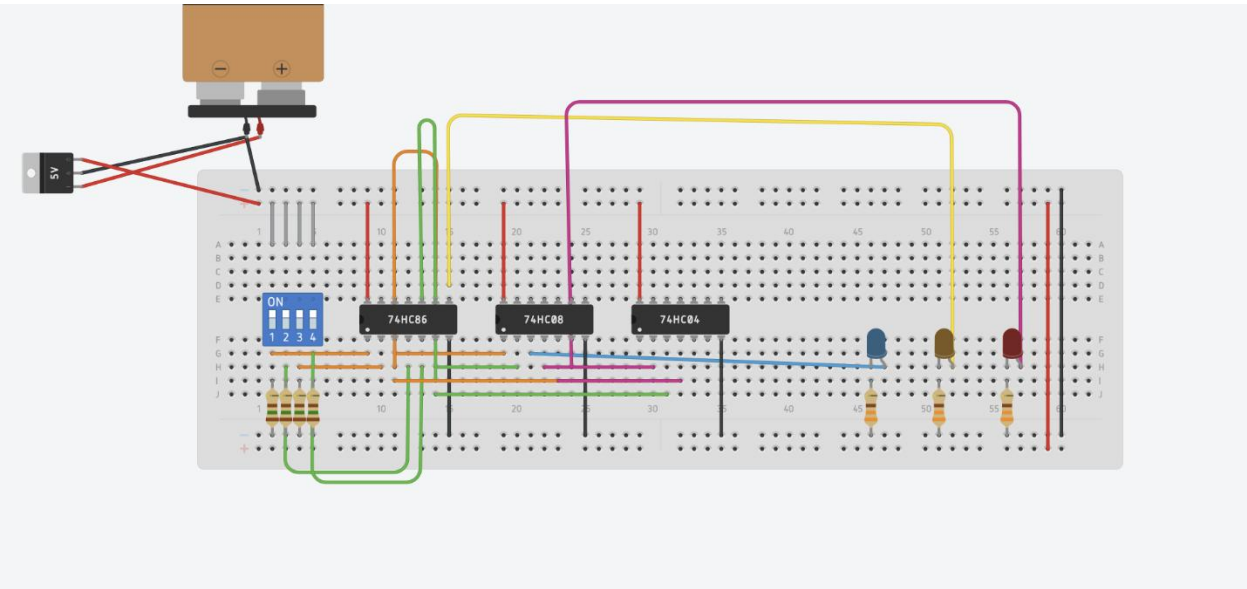


Figure 2: circuit simulation on Tinkercad

Then we started working and implementing the logic circuit on the breadboard directly.

1. Power Distribution and Regulation

    - Voltage Regulation:

      Connected a 9V battery to the power supply module to provide a stable 5V DC supply to the circuit.

    - Breadboard Railing:

      connected the VCC and Ground connections across the breadboard's power rails to get the voltage running in the bread board.

    - Chip Powering:

      Provided power and ground to each Integrated Circuit to their respective VCC and Ground pins

2. Input Configuration

    - DIP Switch Setup: Utilized a 6-position DIP switch to represent the 2-bit inputs.
    - Pull-down Resistors: Wired 150Ω resistors to each switch output for protection.

3. Logic Gate Implementation

    - Bit Comparison (XOR):

      Used the 74LS86 (Quad XOR) to compare individual bits of the two numbers.

    - Equality Detection (AND/NOT):

      Color coded jumper wires were used to Combine the outputs of the XOR gates through 74LS08 (AND) and 74LS04 (Hex Inverter) gates to identify when bits are identical (E=1), differ by one bit (D=1) or differ by two bits(F=1).

4. Output

    - LED bulbs: Connected three LEDs (Green, Yellow, and Red) to represent the F, D, and E outputs and connected them to 330 Ω resistors for protection.

Physical Circuit Assembly

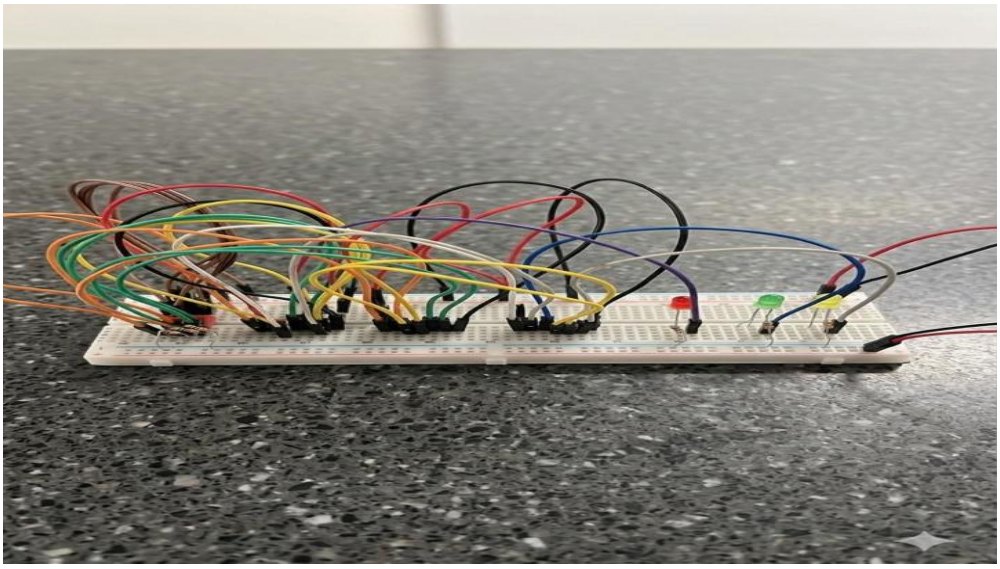


Figure 3: first attempt

Our first attempt had a disorganized layout and incorrect wiring and packing. There were also logical errors between the XNOR and AND gates that disrupted the logic flow. These issues resulted in a non-functional circuit where the LEDs failed to light up for any input combination.

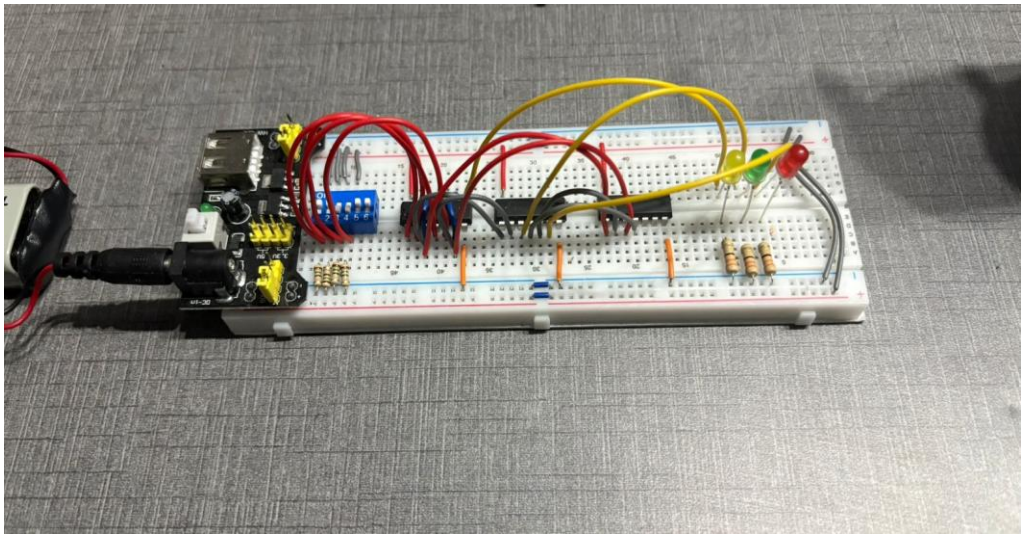
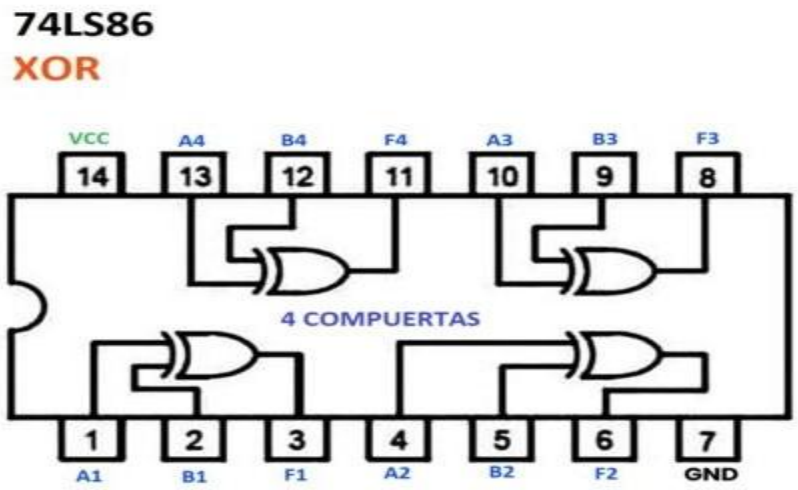


Figure 4: second attempt

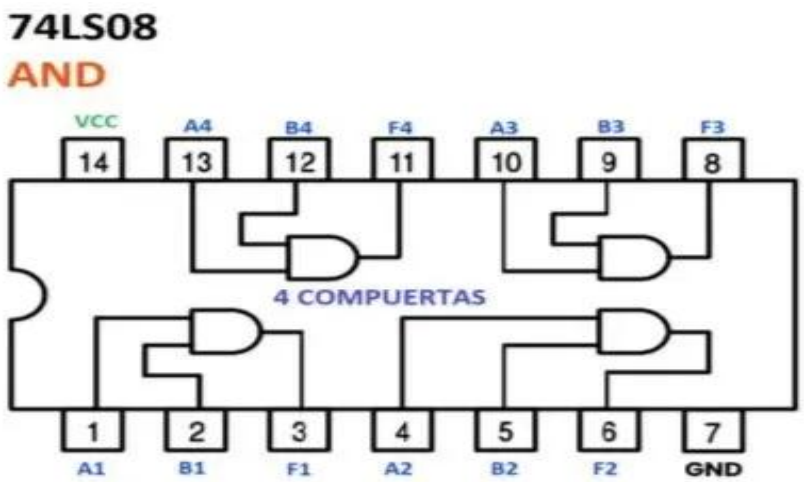
On our second attempt we used shorter, color-coded wiring to resolve previous disorganization issues. We corrected the logic paths between the ICs to ensure exact alignment with our Boolean expressions. This resulted in a fully functional circuit with all LEDs accurately reflecting the needed outputs.

Datasheets used:

- XOR Datasheet:



- AND Datasheet:



- HEX inverter/NOT Datasheet: